

Databases Laboratory 3

Efe Kardes

Emir Avni Sahin

Gafur Samet Saglik

November 29, 2025

Assignment 1: Design a database for StackOverflow

This assignment covers the design and implementation of a database subset for the Q&A website StackOverflow.

a) Entity-Relationship Model (ERM)

We identified 4 main entity types and their relationships based on the requirements.

Entities and Attributes

- **User:** UserID (Key), Email, Name (Composite: First/Last), Languages (Multivalued).
- **Question:** QuestionID (Key), Title, Body, VoteCount (Derived).
- **Answer:** AnswerID (Key), Body, IsAccepted.
- **Tag:** TagID (Key), TagName.

Relationships

- **User POSTS Question (1:N):** One user can post many questions. Total participation on Question side.
- **User WRITES Answer (1:N):** One user can write many answers. Total participation on Answer side.
- **Question HAS Answer (1:N):** One question can have many answers. Total participation on Answer side.
- **Question LABELED_BY Tag (M:N):** Many-to-Many relationship between Questions and Tags.

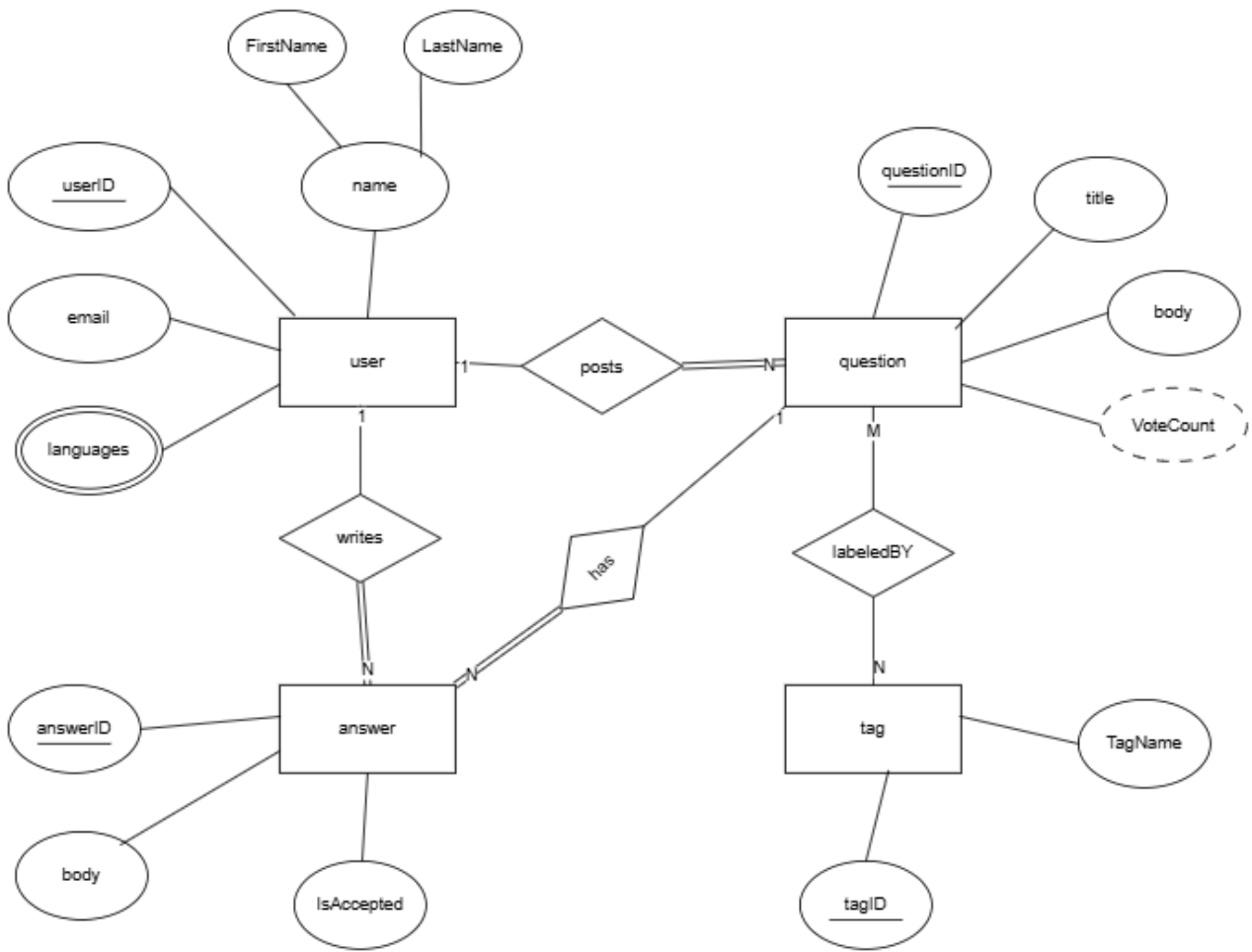


Figure 1: ERM for the StackOverflow database (Chen Notation).

b) Relational Model (RM)

We translated the ERM into a Relational Schema in 3rd Normal Form.

- Composite attributes (Name) are flattened.
- Multivalued attributes (Languages) are moved to a separate table.
- M:N relationships (Question-Tag) are resolved using a junction table.

Schema Definition

- **Users** (UserID, FirstName, LastName, Email)
- **Tags** (TagID, TagName)
- **Questions** (QuestionID, Title, Body, VoteCount, *UserID_FK*)
- **Answers** (AnswerID, Body, IsAccepted, *UserID_FK*, *QuestionID_FK*)
- **User_Languages** (*UserID_FK*, Language)
- **Question_Tags** (*QuestionID_FK*, *TagID_FK*)


```

27     Body TEXT NOT NULL,
28     IsAccepted BOOLEAN DEFAULT FALSE,
29     UserID_FK INT NOT NULL REFERENCES Users(UserID),
30     -- Cascade delete: If Question is deleted, Answers are deleted
31     QuestionID_FK INT NOT NULL REFERENCES Questions(QuestionID) ON DELETE
        CASCADE
32 );
33
34 -- Table for Multivalued Attribute (Languages)
35 CREATE TABLE User_Languages (
36     UserID_FK INT REFERENCES Users(UserID) ON DELETE CASCADE,
37     Language VARCHAR(50),
38     PRIMARY KEY (UserID_FK, Language)
39 );
40
41 -- Junction Table for M:N Relationship
42 CREATE TABLE Question_Tags (
43     QuestionID_FK INT REFERENCES Questions(QuestionID) ON DELETE CASCADE,
44     TagID_FK INT REFERENCES Tags(TagID) ON DELETE CASCADE,
45     PRIMARY KEY (QuestionID_FK, TagID_FK)
46 );

```

Part (d): Data Manipulation (Insert & Delete)

This section demonstrates how to insert data into the database and how the ON DELETE CASCADE constraint functions when deleting data.

Inserting Data

Data must be inserted into parent tables first to satisfy foreign key constraints.

Listing 2: Inserting Data into StackOverflow Tables

```

1  -- 1. Insert a User (Parent)
2  INSERT INTO Users (FirstName, LastName, Email)
3  VALUES ('Jane', 'Doe', 'jane.doe@example.com');
4
5  -- 2. Insert a Tag (Parent)
6  INSERT INTO Tags (TagName)
7  VALUES ('SQL');
8
9  -- 3. Insert a Question (Child - links to User 1)
10 -- Note: UserID_FK = 1 refers to the user created above
11 INSERT INTO Questions (Title, Body, UserID_FK)
12 VALUES ('How do I join tables?', 'I am confused about INNER JOIN vs LEFT
        JOIN.', 1);
13
14 -- 4. Insert an Answer (Dependent - links to User 1 and Question 1)
15 INSERT INTO Answers (Body, UserID_FK, QuestionID_FK)
16 VALUES ('Use INNER JOIN to get matching rows from both.', 1, 1);
17
18 -- 5. Link the Question to the Tag (Junction Table)
19 INSERT INTO Question_Tags (QuestionID_FK, TagID_FK)
20 VALUES (1, 1);

```

Deleting Data (Cascade Demo)

Deleting a question automatically removes its associated answers and tag links.

Listing 3: Demonstrating Cascade Delete

```
1  -- Deleting the Question will automatically delete the related Answer
2  -- and the entry in Question_Tags.
3  DELETE FROM Questions WHERE QuestionID = 1;
```

Part (e): Triggers

This section implements a database trigger to handle the derived attribute `VoteCount`.

Use Case

The `Questions` table contains a `VoteCount` column, which is a derived attribute representing the sum of all upvotes (+1) and downvotes (-1). Manually updating this column every time a vote occurs is inefficient and error-prone. A trigger is used to automate this calculation.

Trigger Logic

We created a `Votes` table to store individual votes. An `AFTER` trigger is defined on this table.

- **Event:** The trigger fires after any `INSERT`, `UPDATE`, or `DELETE` on the `Votes` table.
- **Action:** It executes a function that sums the `Value` of all votes for the specific `Question ID` and updates the `VoteCount` column in the `Questions` table.

Listing 4: Trigger Implementation for Vote Counting

```
1  -- 1. Create Votes Table
2  CREATE TABLE Votes (
3      VoteID SERIAL PRIMARY KEY,
4      Value INT CHECK (Value IN (1, -1)),
5      QuestionID_FK INT REFERENCES Questions(QuestionID) ON DELETE CASCADE
6  );
7
8  -- 2. Create Function to Recalculate Votes
9  CREATE OR REPLACE FUNCTION update_vote_count()
10 RETURNS TRIGGER AS $$
11 DECLARE
12     target_q_id INT;
13 BEGIN
14     -- Check operation type to get the correct ID
15     IF (TG_OP = 'DELETE') THEN
16         target_q_id := OLD.QuestionID_FK;
17     ELSE
18         target_q_id := NEW.QuestionID_FK;
19     END IF;
20
21     -- Update the Question's VoteCount
22     UPDATE Questions
23     SET VoteCount = (
24         SELECT COALESCE(SUM(Value), 0)
25         FROM Votes
26         WHERE QuestionID_FK = target_q_id
27     )
28     WHERE QuestionID = target_q_id;
29
```

```

30     IF (TG_OP = 'DELETE') THEN
31         RETURN OLD;
32     ELSE
33         RETURN NEW;
34     END IF;
35 END;
36 $$ LANGUAGE plpgsql;
37
38 -- 3. Create Trigger
39 CREATE TRIGGER trg_update_vote_count
40 AFTER INSERT OR UPDATE OR DELETE ON Votes
41 FOR EACH ROW
42 EXECUTE FUNCTION update_vote_count();

```

Assignment 2: Flight Data

In this assignment, we imported a flight dataset and analyzed it using SQL queries.

a) Shortest route starting at HAM

```
1 SELECT route_id, route_distance
2 FROM routes
3 WHERE airport_start = 'HAM'
4 ORDER BY route_distance ASC
5 LIMIT 1;
```

	route_id character varying (10)	airport_start character varying (10)	airport_destination character varying (10)	route_distance integer
1	HAM_BRE	HAM	BRE	103

Figure 3: Result for 2a: The shortest route is HAM_BRE (103 km).

b) Destination city of the shortest route

```
1 SELECT a.name
2 FROM routes AS r
3 JOIN airports AS a ON r.airport_destination = a.airport_code
4 WHERE r.airport_start = 'HAM'
5 ORDER BY r.route_distance ASC
6 LIMIT 1;
```

	name character varying (40)
1	Bremen

Figure 4: Result for 2b: The destination city is Bremen.

c) Direct flights to Africa from Hamburg

```
1 SELECT r.route_id, dest_airport.name, dest_airport.continent
2 FROM routes AS r
3 JOIN airports AS start_airport ON r.airport_start = start_airport.
   airport_code
4 JOIN airports AS dest_airport ON r.airport_destination = dest_airport.
   airport_code
5 WHERE start_airport.name = 'Hamburg'
6 AND dest_airport.continent = 'Afrika';
```

	route_id character varying (10) 🔒	name character varying (40) 🔒	continent character varying (10) 🔒
1	HAM_MIR	Monastir	Afrika
2	HAM_AGA	Agadir	Afrika
3	HAM_HRG	Hurghada	Afrika
4	HAM_TUN	Tunis	Afrika
5	HAM_NDR	Nador	Afrika
6	HAM_DJE	Djerba	Afrika

Figure 5: Result for 2c: Direct flights to African destinations.

d) Passengers from HAM to BRU in 2024

```

1 SELECT SUM(f.total_nb_passengers) AS total_passengers
2 FROM flights AS f
3 JOIN routes AS r ON f.route = r.route_id
4 WHERE r.airport_start = 'HAM'
5       AND r.airport_destination = 'BRU'
6       AND f.year = 2024;

```

	total_passengers bigint 🔒
1	71883

Figure 6: Result for 2d: Total passengers = 71,883.

e) Seats from HAM to BRU in 2024

```

1 SELECT SUM(f.nb_flights * f.nb_seats) AS total_seats
2 FROM flights AS f
3 JOIN routes AS r ON f.route = r.route_id
4 WHERE r.airport_start = 'HAM'
5       AND r.airport_destination = 'BRU'
6       AND f.year = 2024;

```

	total_seats bigint 🔒
1	108626

Figure 7: Result for 2e: Total seats = 108,626.

f) Seat occupancy rate (HAM to BRU)

```
1 SELECT
2     (SUM(f.total_nb_passengers) * 100.0 / SUM(f.nb_flights * f.nb_seats)) AS
3     occupancy_rate
4 FROM flights AS f
5 JOIN routes AS r ON f.route = r.route_id
6 WHERE r.airport_start = 'HAM'
7     AND r.airport_destination = 'BRU'
8     AND f.year = 2024;
```

	occupancy_rate	
	numeric	
1	66.1747647892769687	

Figure 8: Result for 2f: Occupancy rate $\approx 66.17\%$.

g) Bonus: Shortest distance HAM to Cape Town

We used a self-join on the routes table to find a connection with one stopover.

```
1 SELECT
2     r1.airport_destination AS stopover_airport,
3     (r1.route_distance + r2.route_distance) AS total_distance
4 FROM routes AS r1
5 JOIN routes AS r2 ON r1.airport_destination = r2.airport_start
6 WHERE r1.airport_start =
```

	stopover_airport	total_distance
	character varying (10)	integer
1	MUC	9779

Figure 9: Result for 2g: Shortest path to Cape Town.

Task 3

a) Top 25 Countries by Population in 2024

What it does: Selects the 25 most populated countries in 2024 using the “Population mid-year estimates” series.

```
SELECT area_name, value AS population_2024
FROM country
WHERE series = 'Population mid-year estimates (millions)'
      AND year = 2024
ORDER BY value DESC
LIMIT 25;
```

b) Does every country have population data for 2024?

What it does: Counts how many countries exist in total and how many have 2024 population values.

```
SELECT
    (SELECT COUNT(DISTINCT area_name)
     FROM country
     WHERE series = 'Population mid-year estimates (millions)') AS
        total_countries,

    (SELECT COUNT(DISTINCT area_name)
     FROM country
     WHERE year = 2024
           AND series = 'Population mid-year estimates (millions)') AS
        countries_with_2024_data;
```

c) Rank Countries by Population Density

What it does: Ranks all entries (ties get the same rank) based on population density using DENSE_RANK().

```
SELECT
    area_name,
    year,
    value AS population_density,
    DENSE_RANK() OVER (ORDER BY value DESC) AS dense_rank
FROM country
WHERE series = 'Population density'
ORDER BY population_density DESC;
```

d) Verify Global Energy Totals (area_id = 1)

What it does: 1. Chooses “appropriate” energy series 2. Sums all country values (except global row) 3. Compares sums vs area_id = 1 4. Checks if deviation is within 1%

```
WITH appropriate_series AS (  
    SELECT 'Primary energy production (petajoules)' AS series  
    UNION ALL SELECT 'Net imports [Imports - Exports - Bunkers] (  
        petajoules)'  
    UNION ALL SELECT 'Total supply (petajoules)'  
) ,
```

```
country_sums AS (  
    SELECT  
        e.year ,  
        e.series ,  
        SUM(e.value) AS summed_value  
    FROM energy e  
    JOIN appropriate_series s ON e.series = s.series  
    WHERE e.area_id <> 1  
    GROUP BY e.year , e.series  
) ,
```

```
global_values AS (  
    SELECT  
        e.year ,  
        e.series ,  
        e.value AS global_value  
    FROM energy e  
    JOIN appropriate_series s ON e.series = s.series  
    WHERE e.area_id = 1  
)
```

```
SELECT  
    c.year ,  
    c.series ,  
    ROUND(c.summed_value , 3) AS summed_countries ,  
    g.global_value ,  
    ROUND(ABS(c.summed_value - g.global_value) / g.global_value *  
        100 , 3)  
        AS deviation_percent ,  
    CASE  
        WHEN ABS(c.summed_value - g.global_value) / g.global_value  
            <= 0.01  
        THEN 'OK ( 1 % )'  
        ELSE 'NOT OK (>1%)'  
    END AS check_status  
FROM country_sums c  
JOIN global_values g
```

```
    ON c.year = g.year AND c.series = g.series
ORDER BY c.year, c.series;
```

e) Missing Data Check Using Cross Join

What it does: Builds every possible combination of (area, year, series). Then finds which entries are missing in the table.

```
WITH all_areas AS (
    SELECT DISTINCT area_id FROM country
),
all_years AS (
    SELECT DISTINCT year FROM country
),
all_series AS (
    SELECT DISTINCT series FROM country
),
all_combinations AS (
    SELECT a.area_id, y.year, s.series
    FROM all_areas a
    CROSS JOIN all_years y
    CROSS JOIN all_series s
)
SELECT
    ac.area_id,
    ac.year,
    ac.series
FROM all_combinations ac
LEFT JOIN country c
    ON c.area_id = ac.area_id
    AND c.year = ac.year
    AND c.series = ac.series
WHERE c.value IS NULL
ORDER BY ac.area_id, ac.year, ac.series;
```

Assignment 4 Part 1: Normalization

a) Analysis of Country & Energy Data Structure

Normal Form & Structure

The provided dataset is in **First Normal Form (1NF)** as it contains only atomic values, but it specifically utilizes an **Entity-Attribute-Value (EAV)** model. This "vertical" structure stores attribute names (like "Solar Power") in rows rather than columns.

Advantages & Disadvantages

The primary advantage of this structure is **flexibility and storage efficiency** for sparse data. New energy metrics can be added indefinitely without altering the database schema, and countries missing specific data points do not require storage-wasting NULL values. However, the major disadvantage is **query complexity and data integrity**. Retrieving multiple attributes for a single country requires complex self-joins or pivoting, and the generic **value** column prevents strict data typing (like mixing integers and decimals), which weakens data validation compared to a standard column-based structure.

Use Cases

This structure is best suited for **scientific or statistical datasets** (like UN data or medical records) where the number of possible attributes is vast, volatile, and highly sparse, making a fixed-column table impractical.

b) Normalization of "Recipes" Relation

1. First Normal Form (1NF)

Requirement: Eliminate non-atomic values. We split the **allergens** column into atomic values. This creates redundancy where ingredient data repeats for each allergen.

ID	Dish	Ingred.	Qty	Cals	Allergen	Cat	Sub
984	Labskaus	egg	1	600	egg	main	regional
984	Labskaus	egg	1	600	seafood	main	regional
984	Labskaus	potato	4	600	egg	main	regional
984	Labskaus	potato	4	600	seafood	main	regional
984	Labskaus	herring	150g	600	egg	main	regional
984	Labskaus	herring	150g	600	seafood	main	regional
984	Labskaus	red beet	100g	600	egg	main	regional
984	Labskaus	red beet	100g	600	seafood	main	regional
984	Labskaus	beef	200g	600	egg	main	regional
984	Labskaus	beef	200g	600	seafood	main	regional
635	Gelato	milk	1L	1400	milk	dessert	ice cream
635	Gelato	strawb.	400g	1400	milk	dessert	ice cream
113	Bob Ch.	beans	2 cups	200	none	main	regional
113	Bob Ch.	tomato	2	200	none	main	regional
113	Bob Ch.	onion	1	200	none	main	regional
113	Bob Ch.	carrot	1	200	none	main	regional

Table 1: Relation in First Normal Form (1NF)

2. Second Normal Form (2NF)

Requirement: Remove partial dependencies. We split the relation into three tables based on the Primary Key dependencies.

Table 2: Dishes (PK: dish_id)

dish_id	dish_name	calories	category	subcategory
984	Labskaus	600 kcal	main	regional
635	Gelato di fragola	1400 kcal	dessert	ice cream
113	Bob Chorba	200 kcal	main	regional

Table 3: Ingredients (PK: dish_id, ingredient)

dish_id	ingredient	quantity
984	egg	1
984	potato	4
984	herring	150g
984	red beet root	100g
984	beef	200g
635	milk	1L
635	strawberries	400g
113	white beans	2 cups
113	tomato	2
113	onion	1
113	carrot	1

Table 4: Allergens (PK: dish_id, allergen)

dish_id	allergen
984	egg
984	seafood
635	milk
113	none

3. Third Normal Form (3NF)

Requirement: Remove transitive dependencies. In the Dishes table, **subcategory** determines **category**. This is extracted to a new table.

Table 5: Categories (New)

subcategory (PK)	category
regional	main
ice cream	dessert

Table 6: Dishes (Updated)

dish_id (PK)	dish_name	calories	subcategory (FK)
984	Labskaus	600 kcal	regional
635	Gelato di fragola	1400 kcal	ice cream
113	Bob Chorba	200 kcal	regional

Table 7: Ingredients (Unchanged from 2NF)

dish_id	ingredient	quantity
984	egg	1
984	potato	4
984	herring	150g
984	red beet root	100g
984	beef	200g
635	milk	1L
635	strawberries	400g
113	white beans	2 cups
113	tomato	2
113	onion	1
113	carrot	1

Table 8: Allergens (Unchanged from 2NF)

dish_id	allergen
984	egg
984	seafood
635	milk
113	none

Assignment 4 Part 2: Determine Primary Key

a) Relation (A, B, C, D, E, F)

Dependencies: $A \rightarrow B, C$; $C \rightarrow D, E$; $A, C \rightarrow F$; $D \rightarrow E$

- **Primary Key:** A
- **Reasoning:** A determines B and C. Through transitivity ($C \rightarrow D, E$), A also determines D and E. With A and C known, F is determined. Thus, $\{A\}^+ = \{A, B, C, D, E, F\}$.

b) Relation (A, B, C, D, E, F)

Dependencies: $A, B, C \rightarrow D$; $A, B, C \rightarrow E$; $A, B, C \rightarrow F$

- **Primary Key:** (A, B, C)
- **Reasoning:** The combination of A, B, C is required to determine all other attributes. No subset of this key determines the non-key attributes.

c) Relation (A, B, C, D, E, F)

Dependencies: $C, D \rightarrow E, F$; $F \rightarrow A, B$

- **Primary Key:** (C, D)
- **Reasoning:** C, D determine E and F. Since F determines A and B, the closure of $\{C, D\}$ includes all attributes.

d) Normal Form Identification

- **Relation (a): 2nd Normal Form (2NF).** The PK is atomic (A), so there are no partial dependencies. However, it violates 3NF due to transitive dependencies (e.g., $A \rightarrow C \rightarrow D$).
- **Relation (b): 3rd Normal Form (3NF).** The PK is (A, B, C). All non-key attributes (D, E, F) fully depend on the key, and there are no dependencies between non-key attributes.
- **Relation (c): 2nd Normal Form (2NF).** The PK is (C, D). There are no partial dependencies. It violates 3NF because of the transitive dependency $F \rightarrow A, B$ (where F is a non-key attribute).

e) 3NF Decomposition for Relation (a)

To achieve 3NF, we decompose to remove transitive dependencies:

1. **Isolate $D \rightarrow E$:** Create table $R_1(D, E)$.
2. **Isolate $C \rightarrow D$:** Create table $R_2(C, D)$.
3. **Remaining Dependencies ($A \rightarrow B, C, F$):** Create table $R_3(A, B, C, F)$.

Final Schema: $R_1(\underline{D}, E)$, $R_2(\underline{C}, D)$, $R_3(\underline{A}, B, C, F)$.